

ELI — Architecture

ELI v2.3.18

August 2026

Contents

Blueprint — ELI MKXI Architecture	1
0. Design principles (the non-negotiables)	2
1. Repository map (recursive LOC, role)	2
2. Entry points & launch	3
3. The request lifecycle (the spine)	4
4. Routing layer (eli/execution)	4
5-6. Agent Bus & the agents (eli/cognition/agent_bus.py)	5
7. The Orchestrator — full 12-stage (eli/cognition/orchestrator.py)	5
8. Inference layer (eli/cognition)	6
9. Grounding & anti-confabulation spine (eli/runtime + eli/core/grounding.py)	6
10. Execution layer (eli/execution/executor_enhanced.py)	7
11. Memory subsystem (eli/memory)	7
12. Perception (eli/perception)	7
13. Persona system (eli/cognition/persona*)	8
14. Daemons & background loops	8
15. Learning / self-training (eli/learning)	8
16. EliWorld (eli/world, eli/gui EliWorld tab, kernel/world_model.py)	8
17. Core infrastructure (eli/core)	8
18. GUI (eli/gui)	9
19. On-disk layout (artifacts/, runtime)	9
20. Known seams & weak points (honest)	9
21. Where to make a change (quick index)	9
Update — 2.3.7 (new subsystems)	10

Blueprint — ELI MKXI Architecture

The full structural/architectural map of ELI MKXI. Every claim here is grounded in the actual source tree (file paths are real and clickable). Where something is observed-at-runtime rather than read-from-code it is marked (*runtime*).

ELI is a **local-first, offline-by-default, model-agnostic** cognitive runtime + assistant GUI (and, as of 2026-06-28, a self-hosted web app — `api/server.py`, documented in `ELI_USER_MANUAL.md`). No cloud, no APIs on the inference path, no hardcoded model. ~160k LOC across 397 Python files (+ `api/server.py`); 223 capabilities (2026-08-21).

0. Design principles (the non-negotiables)

1. **Local-first / offline-by-default.** All outbound network is fail-closed at the socket boundary (eli/core/netguard.py); networked features opt in.
2. **Model-agnostic.** No model name/size is hardcoded on the inference path; ELI discovers and loads whatever GGUF exists. The model is swappable substrate, not a constant.
3. **Grounded / anti-confabulation.** A large dedicated subsystem exists purely to stop a local model stating things it can't back up (see §9).
4. **Emergent persona.** ELI has a distinctive voice; persona drift/banter is intended. Functional bugs are fixed; the personality is not scripted away.

1. Repository map (recursive LOC, role)

Per-package files / LOC measured 2026-07-28.

Package	LOC	Files	Role
eli/runtime	30.4k	94	Grounding spine, evidence, re-sponse/introspection surfaces, daemons, self-improvement
eli/execution	24.2k	15	Router + executor: intent → action → side effects (executor god-file)
eli/gui	22.7k	24	PySide6 desktop app, panels, startup/first-boot wizard, animated face
eli/cognition	15.4k	31	Agent bus, 12-stage orchestrator, inference, persona, reasoning, tone/emotion adaptor, emotional timeline
eli/kernel	14.7k	8	CognitiveEngine (the orchestrating core), scheduler, task bus
eli/perception	8.3k	23	Vision (VL), STT (whisper), TTS, wake word , voice/tone , OS control, gaze
eli/core	8.1k	28	netguard, paths, settings, hardware profile, model download, full_control

Package	LOC	Files	Role
eli/tools	7.4k	29	Image engine, news, registry/capabilities, document tools
eli/memory	7.4k	13	SQLite + FTS5 + FAISS + knowledge graph + working memory
eli/planning	4.0k	24	Proactive daemon, habit scheduler, job/proposal/attention queues
eli/learning	3.4k	12	LoRA self-training pipeline (Phi-3 base), dataset build/eval
eli/coding	2.1k	12	CodeAgent — plan→search→verify→repair coding pipeline
eli/plugins	2.0k	26	Plugin manager + bundled plugins
eli/world	1.6k	26	EliWorld event bus, local world bridge, avatar/ontology/face
eli/integrations	1.1k	9	Optional Ollama client + integration adapters (not on the default path)
eli/utils	1.0k	3	logging, shared helpers
eli/contracts	0.7k	3	typed pipeline contracts
eli/system	0.3k	2	system-level helpers
eli/cli	0.1k	2	headless REPL
total (listed)	~155k	383	full eli/ tree: ~160k / 397 incl. small top-level pkgs

The four god-files (refactor targets — see §20)

File	LOC
eli/execution/executor_enhanced.py	14951
eli/kernel/engine.py	13841
eli/gui/eli_pro_audio_gui_v2_0.py	12100
eli/execution/router_enhanced.py	7621
eli/runtime/deterministic_grounding_gate.py	4301
eli/memory/memory.py	4734

2. Entry points & launch

Command	Target	Notes
eli, eli-mkxi, eli-cli	eli.gui.app:main	GUI (default)
eli-gui	eli.gui.app:main	GUI script
python -m eli	eli/__main__.py	dispatches to GUI or headless
python -m eli --headless / -H	eli.cli.headless:run_headless	terminal REPL, no Qt
./eli.sh	python -m eli	venv launcher
bash install.sh	—	venv + deps + clean config seed + verify

Boot path (GUI): app.py → (first-boot wizard if no model, eli/gui/panels/startup.py) → eli_pro_audio_gui_v2_0.py:main
 builds EliMainWindow → constructs the **CognitiveEngine singleton** → loads GGUF per config/settings.json
 → starts daemons.

3. The request lifecycle (the spine)

Everything funnels through **CognitiveEngine.process()** (eli/kernel/engine.py). Two paths diverge by reasoning mode:

```

user input
  |
  ▼
[Router] eli/execution/router_enhanced.py :: route()
  |   priority pipeline → {action, args, confidence, meta.matched_by}
  ▼
CognitiveEngine.process(action, args, ...)
  |
  ├── PHASE45 fast-path? (deterministic OS/media/status/job actions)
  |   → execute_action() → return VERBATIM (no LLM)           [$10]
  |
  ├── non-quick mode OR forced → Internal Orchestrator (full 12-stage) [$7]
  |
  └── quick mode (default) →
      AgentBus.dispatch() [$6] → bus_result {grounding, agents, ctx}
          |
          ├── Grounding escalation hook [$9] (CHAT + low grounding + factual)
          |   → web tier / local tier / hedge → may RETURN here
          |
          ├── self-contained action? → return executor result verbatim
          ├── grounded control action? → grounded synthesis
          └── CHAT → _build_enhanced_system() → broker.infer()/stream
                  → output_governor → response

```

Pipeline stages logged at runtime: Stage 1 Intent → ... → Stage 12 Confidence. Quick mode short-circuits stages 2-4/HyDE; the orchestrator runs all 12.

4. Routing layer (eli/execution)

- **router_enhanced.py** — route(text) -> {action, args, confidence, meta}. Regex-first **priority pipeline** (“explicit priority pipeline installed”): ordered prepasses (web realtime lookup, media, file/PDF, memory, control...) then a core router, with an LLM-intent fallback (cognition/llm_intent.py). Holds module state like `_last_used_path` for referential follow-ups.

- **execution_planner.py** / **execution_intent_packets.py** — typed ExecutionPlan / RouteDecision artifacts the bus consumes.
- **route_authority.py**, **route_contracts.py**, **tool_execution_authority.py**, **operator_policy.py** — guardrails on what may be routed/executed.
- **portable_intent_contract.py**, **router_plugin_intents.py**, **executor_plugin_handlers.py**, **media_runtime.py**, **operator_actions.py** — plugin/media/OS intent wiring.

Key router behaviours (each is a fixed bug → guarded by tools/eval/cases.yaml): media controls require **whole-word + command-shape** (no substring “tab” in “metabolise”); bare “do a search” needs a subject; meta-questions stay CHAT; realtime facts (“release date”) → WEB_SEARCH.

5-6. Agent Bus & the agents (eli/cognition/agent_bus.py)

AgentBus.dispatch(user_input, intent, ...) -> DispatchResult. Selects an agent set (_select_agents_for_intent, or broad fan-out), runs them over a dependency DAG (falls back to flat parallel), aggregates.

DispatchResult fields: agent_results[], action_result, memory_context, aggregated_confidence (route+grounding blend), **grounding_confidence** (agent-evidence only — the honest “is this backed by anything” signal), agents_used, confidence_label, orchestrator_plan.

14 registered bus agents (_ALL_AGENTS):

Agent	Role
BusMemoryAgent	recall from SQLite/FTS/FAISS into context
KnowledgeGraphAgent	entities/relations from the KG
SystemAgent	runs executor actions (WEB_SEARCH, RUNTIME_AUDIT, ...)
OrchestratorAgent	plan/coordinate grounded synthesis
FileCodeAgent	searches the codebase for code-grounded answers
IntrospectionBusAgent	live runtime/self introspection
CapabilityAgent	what ELI can do (capability manifest)
ReflectionAgent	session reflection/insights
ProactiveAgent	proactive patterns/signals
HabitAgent	learned habits
SelfImprovementAgent	self-improvement state
FrontierAgent	frontier reasoning hooks
PluginAgent	dispatches to registered plugins
VoiceAgent	STT/TTS engine state

+ **the 15th: CodeAgent** (eli/coding/agent.py) — a *separate* coding pipeline (plan → search → verify → repair, beam/DAG), invoked for CODE_SOLVE / GENERATE_SCRIPT, not a bus agent.

7. The Orchestrator — full 12-stage (eli/cognition/orchestrator.py)

Runs for non-quick modes (and forced cases). Stages (as logged):

1. **Intent Routing** → action
2. **Persona Lock** → persona/identity fixed for the turn
3. **HyDE** → hypothetical-doc query expansion (cognition/hyde.py)
4. **Planner** → which retrieval channels + limits for the mode 5/6/7. **Parallel Retrieval** → keyword + **FTS5** (conversation_turns) + **FAISS** vector + RAG + **KG**
5. **Hybrid Merge** → combine channel hits
6. **Rerank** → cognition/reranker.py

7. **Context Assembly** → assembled grounded context 10.5. **Persona Handoff** → persona brief built for generation
8. **LLM Generation** → stream/oneshot via the broker
9. **Confidence** → score vs threshold (PASS/repair)

Quick mode (default) skips 2-4/HyDE and uses the AgentBus directly; the orchestrator is the “deep” path.

8. Inference layer (eli/cognition)

- **inference_broker.py** — the **only canonical inference path**. `broker.infer()`.
- **gguf_inference.py** — llama-cpp wrapper, live runtime params, `chat_completion()`, runtime snapshot publishing. Model-agnostic; loads whatever GGUF is configured.
- **reasoning_modes.py** — modes: quick, chain_of_thought, self_consistency, tree_of_thoughts, constitutional_ai (private execution strategy, not shown raw).
- **chat_model.py**, **context_builder.py**, **context_synthesiser.py**, **user_info_builder.py** — prompt/context assembly.
- **llm_intent.py** — LLM fallback for ambiguous routing. Decoding is constrained by a GBNF grammar built from the live action catalogue, so the model can only name a capability that actually exists and can only emit well-formed JSON — an invented action is unrepresentable rather than rejected after the fact. Backends without grammar support (Ollama, remote) fall back to the free-text path unchanged.
- Token/context budgeting lives in `engine.py::_build_enhanced_system` (budget-aware persona trim) + `core/dynamic_runtime_budget.py`.

9. Grounding & anti-confabulation spine (eli/runtime + eli/core/grounding.py)

ELI’s signature subsystem — keeps a local model from confidently stating unbacked facts.

- **deterministic_grounding_gate.py** (4.3k) — the gate; decides when an answer must be backed by evidence vs may be free CHAT; blocks raw evidence/telemetry packets from leaking to the user.
- **grounding_escalation.py** — *tiered* escalation (built this cycle): a checkable factual turn with **low grounding** escalates — external fact → web agent, self/project fact → broad local agent fan-out — and **hedged honestly** if nothing grounds it. Trigger is grounding, *not* the (often-high) response-confidence. Env: `ELI_GROUNDING_ESCALATION`.
- **evidence_ledger.py**, **evidence_store.py**, **evidence_arbitration.py**, **memory_evidence.py** — what counts as evidence, where it’s recorded, conflict resolution.
- **persistence_gate.py** — stops internal report-dumps/junk being written to memory (and replayed later).
- **response_contracts.py**, **response_packets.py**, **response_policy.py**, **final_response_assembly.py**, **final_response_provider.py**, **user_visible_response_surface.py** — shape/clean the final user-visible answer; format internal “surface packets” into readable text (never raw JSON).
- **truth_report.py**, **eli_identity_audit.py**, **control_contracts.py** — runtime truth evidence, identity grounding, control-action contracts.
- **output_governor.py**, **response_governance.py**, **response_sanitizer.py**, **persona_hygiene.py** (cognition) — final governance/sanitisation pass.

This is also where the **known weak seam** lives: internal-state→spoken-output leaks (identity confabulation, telemetry dumps) — partially defended, the active frontier (see §20).

10. Execution layer (eli/execution/executor_enhanced.py)

- `execute(action, args) -> dict` — **223 dispatch actions**; runtime capability manifest reports **223 capabilities, 199 of them routable** (*live, read from the manifest each boot*).
- **PHASE45 fast-path** (`engine.py`) — deterministic OS/media/status/job actions (`VOLUME`, `MEDIA_CONTROL`, `NEXT_MEDIA`, `OPEN_APP`, `DATE`, `SHELL_EXEC`, `ANALYZE_IMAGE`, `CHECK_JOB`, `BACKGROUND_JOBS`, ...) bypass the LLM and return the executor result **verbatim** — never paraphrased.
- Action families: media/OS control, file/document (`SUMMARIZE_FILE`, `ANALYZE_PDF[_FOLDER]` incl. per-file mode, `CREATE_DOCUMENT`), web (`WEB_SEARCH`), news, weather, memory (`MEMORY_STORE/RECALL`), code (`CODE_SOLVE`, `GENERATE_SCRIPT` → §coding), background jobs, self/runtime reports (`RUNTIME_STATUS`, `SELF_REPORT`, `RUNTIME_AUDIT`, `EXPLAIN_*`), vision, image generation, scheduling/reminders.
- **Background jobs** (`runtime/background_tasks.py`) — submit/get/list; heavy PDF-folder analysis auto-backgrounds; `CHECK_JOB` surfaces the real result.

Plugins (eli/plugins, manager.py)

Bundled: `calendar`, `document_reader`, `media`, `notes`, `pomodoro`, `system_stats`, `tts`, `weather`, `web`, `web_automation`. State in `config/plugins_state.json` (gitignored). Custom agents/plugins are trust-gated.

11. Memory subsystem (eli/memory)

- **memory.py** (`Memory`, `get_memory()`) — long-term store over SQLite + **FTS5**, plus failure/observation/habit logging with anti-junk filters.
- **vector_store.py** — **FAISS** index at `artifacts/vectors/index.faiss`; embedder models/`embeddings/nomic-embed-text-v1.5.Q4_K_M.gguf` (*runtime*).
- **Knowledge graph** — `kg_entities` / `kg_relations` (FTS5-backed) in the user DB.
- **Working memory** (`cognition/working_memory.py`) — session-pinned facts, junk-pin guard; restored across sessions.
- **Persona/personal memory** — `runtime/personal_memory_*`, `persona_updater`.

Two databases (*runtime*): `artifacts/db/user.sqlite3` (user-facing) and `artifacts/db/agent.sqlite3` (`agent/self-improvement`). Observed tables include: `memories(+fts)`, `conversation_turns`, `conversations`, `kg_entities(+fts)`, `kg_relations`, `recall_log`, `runtime_events`, `news_articles(+fts)`, `news_reflections`, `habits/habit_events/habit_rules`, `observations`, `learning_replay`, `working_memory_pins`, `user_patterns`, `session_summaries`, `corrections`, `failures`, `error_tracking`, `improvements`, `capability_proposals`, `code_patches`, `agent_dispatches`, `agent_metrics`. Retrieval is hybrid: keyword + FTS5 + FAISS + RAG + KG, merged & reranked (§7).

12. Perception (eli/perception)

- **Vision** — `vision.py` (model-agnostic VL: Moondream / Qwen2.5-VL hot-swap; mtmd encoder forced to CPU to avoid CUDA clip segfault), `analyze_image.py`, `ambient_vision.py` (ambient toggle), `screen_locator.py`, `gaze_engine.py`, `analyze_mesh.py`, `extract_equations.py`.
 - **STT** — `local_whisper_stt.py` (faster-whisper `small.en`, CPU int8, local_only when offline), `audio_stt.py`, `eli_listen.py`, `voice_worker*.py` (wake-word “computer”, music-bleed filter, echo gate).
 - **TTS** — `tts_router.py` (Piper voices, e.g. `en_US-amy-medium`).
 - **OS** — `os_controller.py` (`app/window/keyboard/mouse`), `analyze_csv.py`, `analyze_pdfs.py`.
-

13. Persona system (eli/cognition/persona*)

- `persona.txt` (authored) + `persona.auto.txt` (overlay) — the voice.
 - `persona_updater.py` — updates overlay/KG/user-profile each turn (tone signals).
 - `persona.py`, `persona_values.py`, `persona_status.py`, `persona_hygiene.py`.
 - `tone_analyzer.py` — writes tone preference signals (correction/humor/depth).
 - Generation injects a budget-trimmed persona + situation brief (`engine.py::_build_enhanced_system`); a live-runtime fact is injected for model/identity questions so ELI reports the loaded model instead of denying it.
-

14. Daemons & background loops

- **Proactive daemon** (`planning/proactive_daemon.py`) — continuous learning; pattern signals (time habit, topic focus, recurring errors, active project).
 - **Self-improvement loop** (`runtime/self_improvement.py`) — learns from logged failures (now filtered of transient/user errors).
 - **Habit scheduler** (`planning/habits_scheduler.py`, `habits.py`) — learned routines; guarded against junk-rule replay.
 - **Scheduler / task bus** (`kernel/scheduler.py`, `kernel/task_bus.py`).
 - **Background tasks** (`runtime/background_tasks.py`) — async heavy jobs.
 - **Code monitor** (`runtime/code_monitor.py`), **ambient vision loop**.
 - **World event bus** (`world/world_event_bus.py`) — fed confidence/agent events.
-

15. Learning / self-training (eli/learning)

LoRA fine-tuning pipeline on a Phi-3 base: `bootstrap_phi3_base.py`, `base_model_resolver.py`, `dataset_builder.py`, `dataset_filters.py`, `merge_reviewed_datasets.py`, `export_trainable_dataset.py`, `training_preflight.py`, `lora_trainer.py`, `lora_trainer_guard.py`, `lora_eval.py`. Turns reviewed interactions into trainable datasets and adapters.

16. EliWorld (eli/world, eli/gui EliWorld tab, kernel/world_model.py)

An internal/spatial “world” the agent inhabits (rooms like the Reflection Chamber/Anomaly Room appear (*runtime*)). `world_event_bus.py` receives runtime events; `local_world_bridge.py` bridges to the model; snapshots/ledger/journal under `artifacts/world/`. Experimental/creative subsystem.

17. Core infrastructure (eli/core)

- **netguard.py** — offline-by-default: `guarded_urlopen`, process-wide socket guard (fail-closed on non-loopback while offline), `allow_network()` scoped context for deliberate user-initiated fetches (e.g. model download).
- **paths.py**, **portable_paths.py**, **legacy_paths.py**, **db_paths.py** — canonical path resolution (dev vs platformdirs; `ELI_*` overrides).
- **runtime_settings.py**, **config.py** — settings load/merge/heal; DEFAULTS are clean (offline, no model, wizard on). `config/settings.json` is per-user/gitignored, seeded from `config/templates/settings.template.json`.
- **hardware_profile.py**, **startup_hardware_optimizer.py**, **dynamic_runtime_budget.py** — detect GPU/VRAM, pick ctx/gpu_layers/batch.
- **first_run.py**, **first_run_wizard.py** — onboarding state.

- **model_download.py** — curated GGUF downloader (catalog, resumable, GGUF-magic + size validated, netguard-gated). Install-time menu only — inference stays model-agnostic.
- **grounding.py** — `is_grounded_query` etc. (shared grounding helpers).

18. GUI (eli/gui)

PySide6 (LGPL — not PyQt/GPL, see `pyproject.toml`). `eli_pro_audio_gui_v2_0.py` (main window, 11k LOC), `app.py` (entry/boot), `labs_tab.py` (Experimental), panels in `eli/gui/panels/` (`startup.py` = `StartupModelSelectionDialog` + `FirstBootWizard` with curated download, `HardwareTuningDock`). `EliWorld` tab, Operator console, Proactive dock. `qt_compat.py` shim allows PyQt fallback from source.

19. On-disk layout (artifacts/, runtime)

```
artifacts/
├─ db/{user,agent}.sqlite3      # memory + agent state
├─ vectors/index.faiss         # semantic index
├─ conversations/ , conversations/archive/
├─ incidents/<date>.jsonl       # incident log
├─ proactive/ , world/{snapshots,ledger,journal}/
├─ runtime/users/<uuid>/user_profile... # per-user profile
├─ runtime_snapshot.json       # live model/runtime truth
├─ documents/ , scripts/       # generated artifacts
├─ analyze_image_*/            # vision outputs
config/  settings.json(gitignored) · templates/ · plugins_state.json ...
models/  <your>.gguf · embeddings/ · whisper/ · image/
voices/  Piper ONNX
```

20. Known seams & weak points (honest)

1. **God-files** (§1) — four files $\approx$ a third of the codebase; the active refactor target. Split by concern, hold a size ceiling, keep a repair-log.
2. **Internal-state $\rightarrow$ spoken-output seam** — the recurring failure class (identity confabulation, telemetry/packet leaks, ungrounded factual claims). The grounding gate + escalation defend it but don't fully close it on the plain CHAT path. This is *the* frontier item.
3. **Swallowed errors** — many except `Exception: pass`; convert to `logged/raised` so failures surface instead of hiding.
4. **No capability eval until now** — `tools/eval/` (see `tools/eval/README.md`) is the new measurement layer; coverage grows by turning every bug-log into a case.
5. **Latency vs model size** — on an 8GB GPU a 24B at Q5 offloads few layers $\rightarrow$ minutes/reply; a 7-9B Q4 is the sweet spot. The model picker should warn on poor VRAM fit.

21. Where to make a change (quick index)

To change...	Edit...
how input is routed to an action	<code>eli/execution/router_enhanced.py</code>
what an action does	<code>eli/execution/executor_enhanced.py</code>
which agents run / how grounded	<code>eli/cognition/agent_bus.py</code>
the deep 12-stage retrieval	<code>eli/cognition/orchestrator.py</code>

To change...	Edit...
prompt/persona budget, the engine spine anti-confabulation / verify-or-hedge	eli/kernel/engine.py eli/runtime/grounding_escalation.py, deterministic_grounding_gate.py
final answer shaping/cleanup	eli/runtime/*response*, cognition/output_governor.py
memory store/recall	eli/memory/memory.py, vector_store.py
model loading / inference	eli/cognition/gguf_inference.py, inference_broker.py
offline/network policy	eli/core/netguard.py
paths / settings / model download	eli/core/{paths, runtime_settings, model_download}.py
the GUI	eli/gui/eli_pro_audio_gui_v2_0.py, gui/panels/
eval / regression board	tools/eval/ (+ tools/eval/README.md)

“

Update — 2.3.7 (new subsystems)

eli/plugins/ — from a loader to a gated marketplace client

Module	LOC	Role
manager.py	616	discovery, enable/disable, install/uninstall (pre-existing, now data-dir aware)
permissions.py	395	capability vocabulary, consent decisions, grants, audit ledger
manifest.py	302	manifest schema + static capability verification against source
integrity.py	237	sha256 pinning, ed25519 signatures, operator-trusted publishers
security_scan.py	547	11-engine malware scanner
marketplace.py	487	federated registries, preview/install, licence keys
mcp.py	440	MCP server config + runtime preflight + handshake verification

_plugins_dir() now resolves through paths.plugins_dir() rather than returning the source tree; _plugin_search_dirs() scans bundled built-ins **and** user installs, so a downloaded plugin never has to be written into a read-only installation.

eli/cognition/ — agents get a specification

Module	LOC	Role
agent_spec.py	372	AgentSpec: objective, prompt, triggers, success criteria, examples

Module	LOC	Role
agent_trust.py	262	path-keyed trust with provenance, scanning, revocation

agent_bus.SpecAgent runs a spec against the local model and scores the output against the spec's own criteria. Spec agents execute no arbitrary code and therefore bypass the trust chain entirely.

eli/learning/ — the LoRA pipeline becomes usable

Module	LOC	Role
target_registry.py	266	operator-declared targets, any model family
review_queue.py	270	the human review gate the trainer always required

eli/gui/ — three new surfaces

tabs/training_tab.py (Labs ► Training), tabs/marketplace_tab.py (Settings ► Marketplace), panels/permission_dialog.py (consent dialog + worker → GUI-thread bridge).

Paths

paths.learning_dir() joins models_dir() / voices_dir() / plugins_dir() in the dev-vs-data-dir pattern. The rule this all follows: **runtime state never lives in the installation**, because project_root() is a read-only mount on a packaged build.